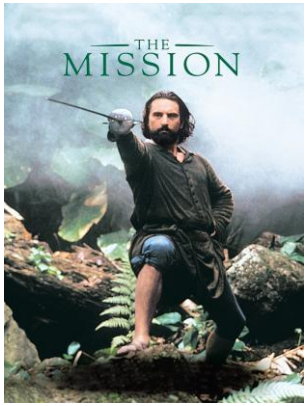
MACHINE LEARNING 2

Ciência de Dados para Negócios

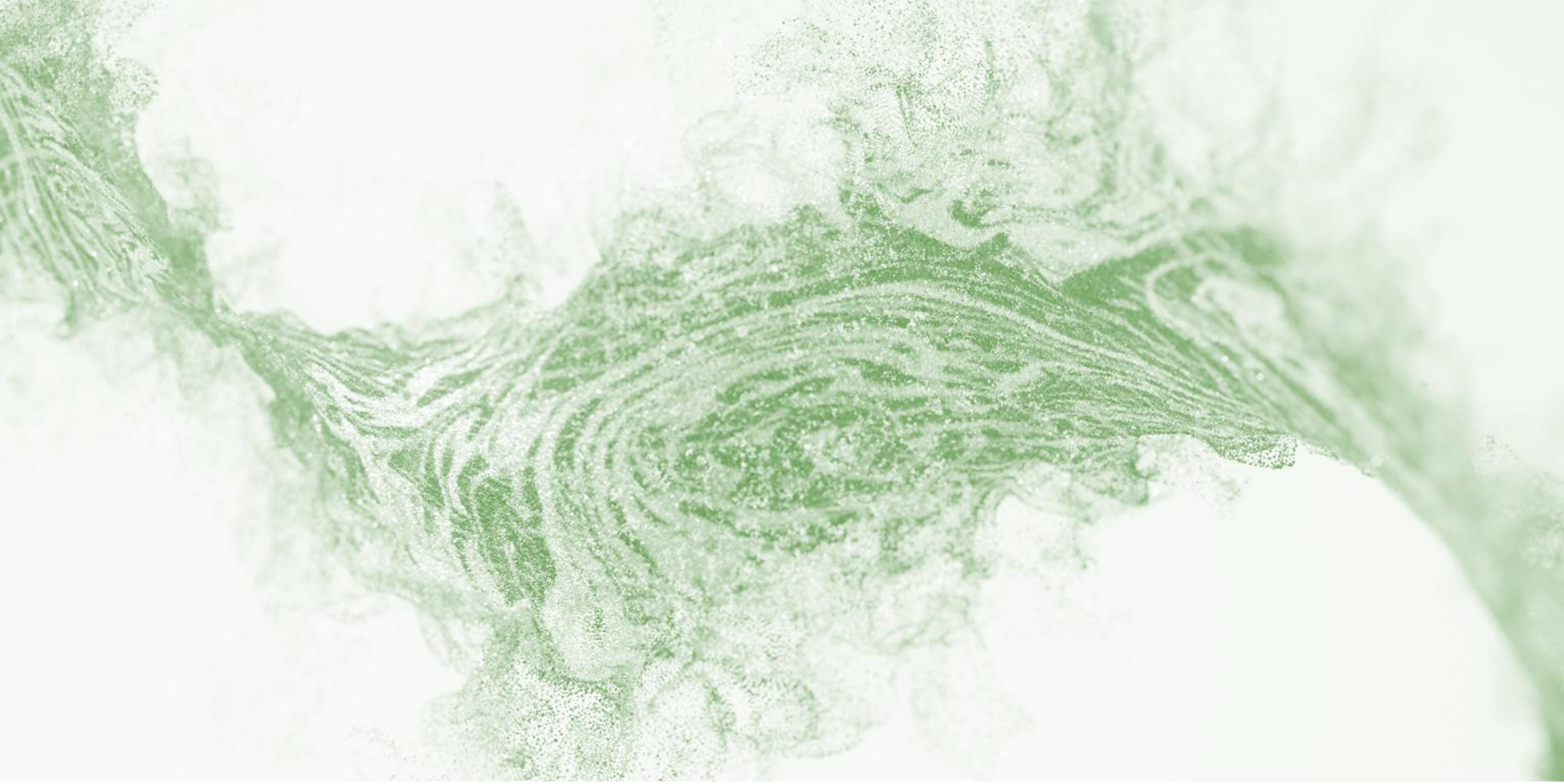
FATEC Votorantim



Prof. Luis Claudio

M. Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)



I – INTRODUÇÃO ÀS REDES NEURAIS ARTIFICIAIS

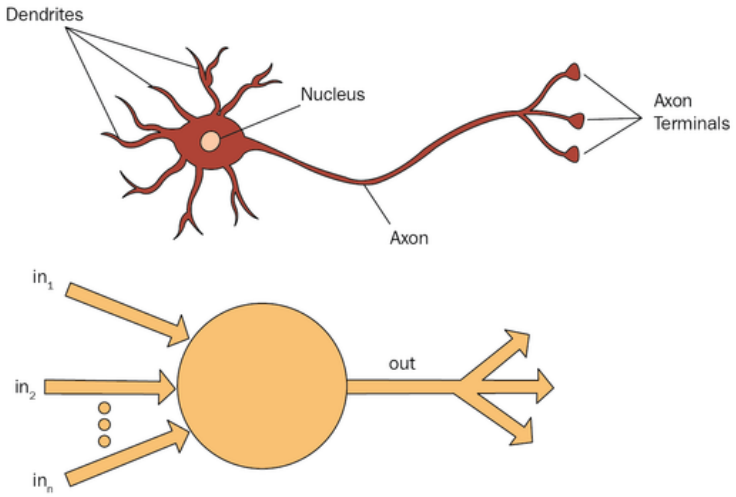
Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

O QUE É UMA REDE NEURAL

Na década de 1940, pesquisadores como **Warren McCulloch** e **Walter Pitts** tentaram responder à seguinte pergunta:

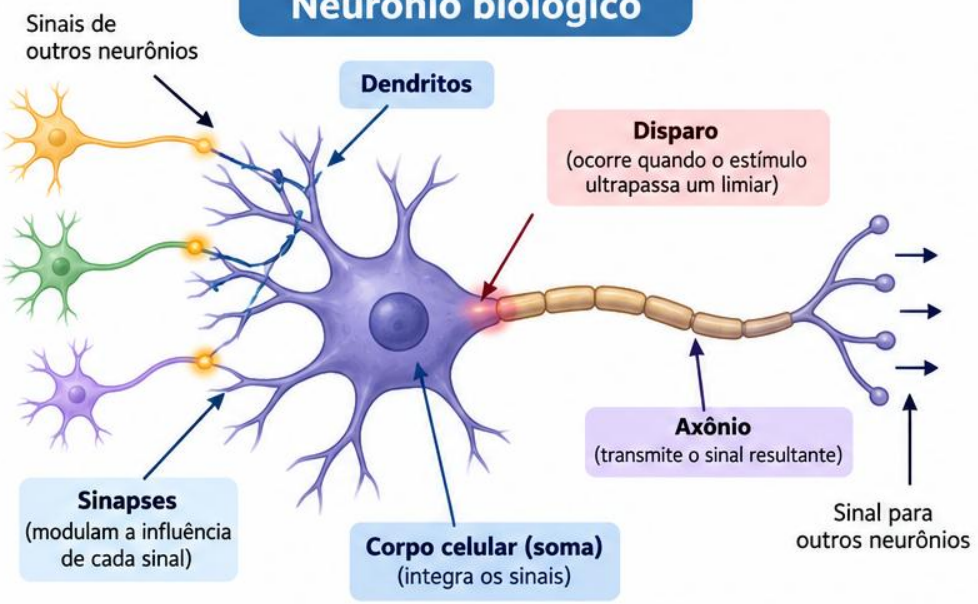
Como o cérebro consegue aprender e processar informações a partir de bilhões de neurônios interconectados?

A ideia foi criar um modelo matemático simplificado do neurônio biológico:

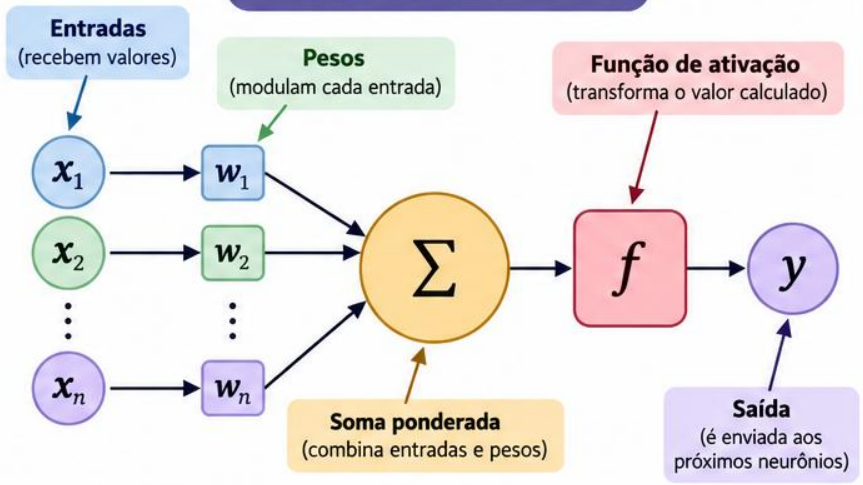


Neurônio biológico	Neurônio artificial
Dendritos recebem sinais de outros neurônios	**Entradas** $(x_1, x_2, \ldots, x_n)$ recebem valores
Sinapses modulam a influência de cada sinal	**Pesos** $(w_1, w_2, \ldots, w_n)$ modulam cada entrada
Corpo celular (soma) integra os sinais	**Soma ponderada** combina entradas e pesos
Disparo ocorre quando o estímulo ultrapassa um limiar	**Função de ativação** transforma o valor calculado
Axônio transmite o sinal resultante	**Saída** (y) é enviada aos próximos neurônios

Neurônio biológico



Neurônio artificial



$$z = \sum_{i=1}^{n} w_i x_i + b \quad \rightarrow \quad y = f(z)$$

NEURÔNIO ARTIFICIAL

O neurônio artificial é a **unidade básica de processamento** de uma rede neural. Ele recebe um conjunto de valores de entrada, realiza uma **combinação linear** **por meio de pesos** (*weights*) e de **um *bias***, e aplica uma **função de ativação** para produzir sua saída.

Matematicamente, o neurônio calcula a entrada líquida:

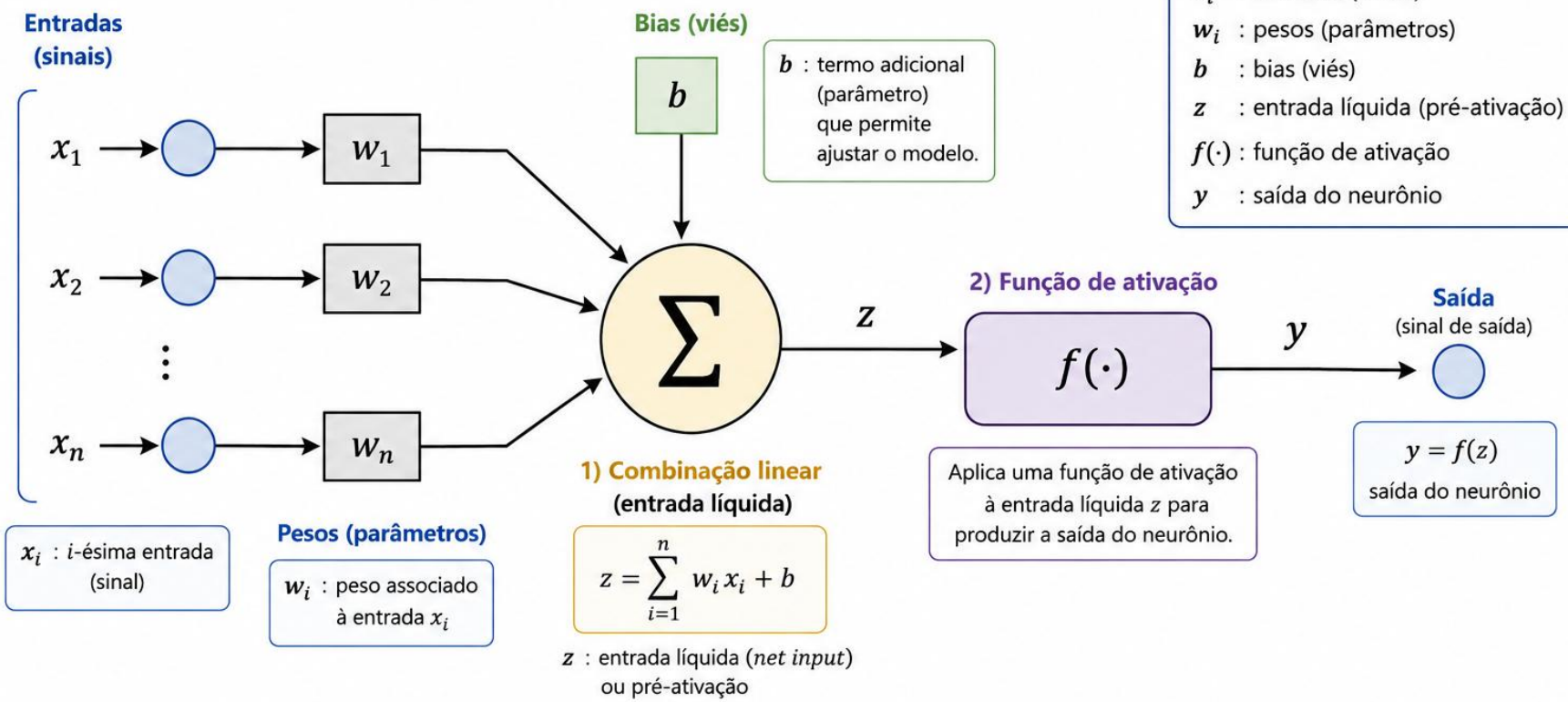
$$z = \sum_{i=1}^{n} w_i x_i + b$$

Em seguida, aplica uma função de ativação:

$$y = f(z)$$

A saída y é então encaminhada para os neurônios da camada seguinte **OU**, caso seja a última camada oculta, utilizada como saída final.

NEURÔNIO ARTIFICIAL



Legenda

- x_i : entradas (sinais)
- w_i : pesos (parâmetros)
- b : bias (viés)
- z : entrada líquida (pré-ativação)
- $f(\cdot)$: função de ativação
- y : saída do neurônio

b : termo adicional (parâmetro) que permite ajustar o modelo.

x_i : i-ésima entrada (sinal)

Pesos (parâmetros)

w_i : peso associado à entrada x_i

1) Combinação linear (entrada líquida)

$$z = \sum_{i=1}^{n} w_i x_i + b$$

z : entrada líquida (*net input*) ou pré-ativação

Aplica uma função de ativação à entrada líquida z para produzir a saída do neurônio.

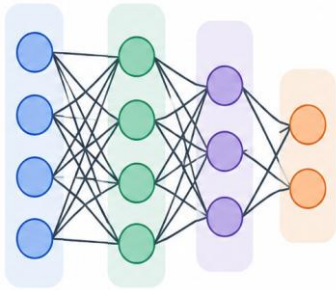
$y = f(z)$
saída do neurônio

Como funciona:

1. Cada entrada x_i é multiplicada pelo seu peso w_i.
2. Os valores ponderados são somados e o bias b é adicionado, produzindo z.
3. A função de ativação $f(\cdot)$ transforma z na saída y.
4. A saída y é enviada para os neurônios da próxima camada ou usada como saída final da rede.

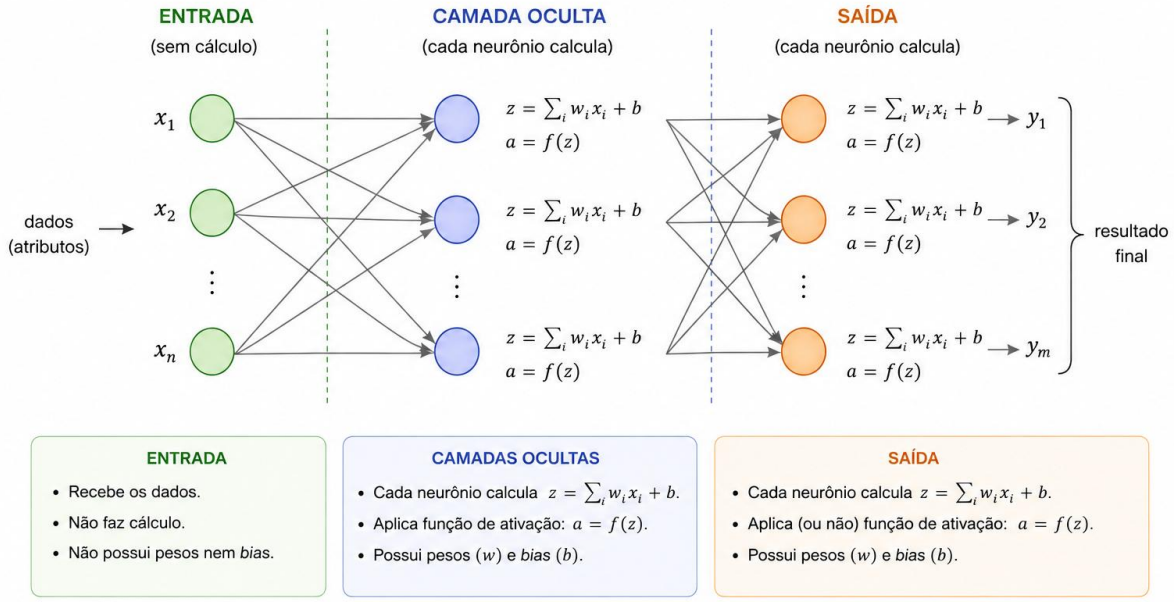
Observação histórica: Em modelos inspirados no neurônio biológico, o valor z já foi chamado de "potencial de ativação". Hoje, os termos mais comuns são entrada líquida, pré-ativação ou simplesmente z.

AS CAMADAS

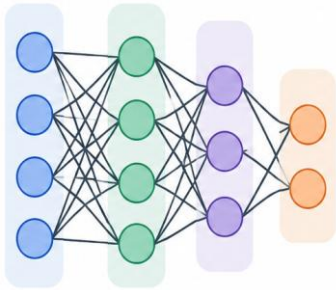


Redes neurais são modelos de **aprendizagem de máquina** compostos por **neurônios artificiais organizados em camadas**, capazes de aprender padrões a partir dos dados.

São compostas por **três tipos de camadas:** camada de **Entrada** (Input), Camadas **Ocultas** (Hidden) e Camada de **Saída** (Output).



A CAMADA DE ENTRADA



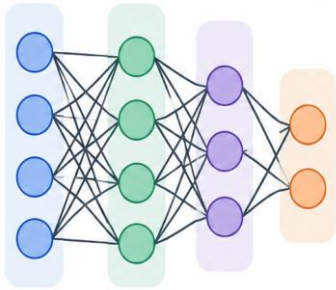
A camada de **Entrada** (*input layers*) recebe os dados de entrada (atributos), representados de **forma numérica** (dados tabulares, pixels de uma imagem etc.).

Em uma MLP tradicional (*), a camada de entrada apenas recebe os dados e os encaminha para a primeira camada oculta. Ela **não realiza processamento matemático** (não possui pesos, bias ou função de ativação).

Por esse motivo, alguns autores **não contabilizam a camada de entrada** ao medir a **profundidade** (*depth*) da rede neural.

* Em arquiteturas mais avançadas, a camada de entrada pode ser seguida por transformações específicas, realizadas por outras camadas, como uma camada de *embedding* em problemas de PLN.

AS CAMADAS OCULTAS



Nas camadas **Ocultas** (*hidden layers*), cada neurônio recebe as saídas da camada anterior, calcula uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$) e aplica uma **função de ativação** para produzir sua saída.

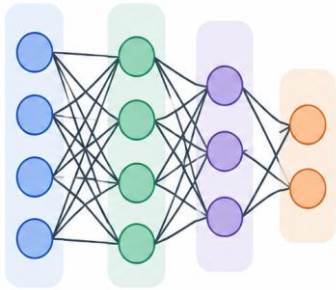
A **quantidade de camadas ocultas** e o **número de neurônios em cada camada** são **hiperparâmetros** definidos durante o projeto da rede.

Não existe uma regra universal para definir a quantidade de neurônios. Algumas **heurísticas** podem ser utilizadas como ponto de partida, como considerar valores próximos à quantidade de nós da camada de entrada ou potências de 2 (16, 32, 64, 128, 256).

As camadas ocultas quase **sempre utilizam funções de ativação**, que introduzem **não linearidade** e permitem que a rede aprenda **representações e relações complexas** a partir dos dados.

Sem funções de ativação nas camadas ocultas, sucessivas transformações lineares equivalem a uma única transformação linear, **limitando a capacidade da rede de modelar relações complexas.**

A CAMADA DE SAÍDA



Na camada de **Saída** (*output layer*) os neurônios também calculam uma combinação linear utilizando **pesos** e **bias** ($z = \sum_{i=1}^{n} w_i x_i + b$). Ou seja, ela **também tem pesos e bias aprendidos durante o treinamento**.

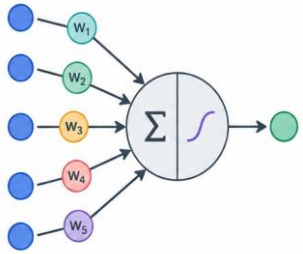
Quando aplicável, a camada de saída também usa **funções de ativação** para o processamento na camada de saída transforma as **representações internas da rede na resposta final do modelo**, adequada ao problema.

A **função de ativação** dependem do tipo de problema, como regressão (identidade), classificação binária (sigmoide) ou multiclasse (softmax).

Em arquiteturas tradicionais*, a **camada de saída é única**, embora possa conter **um ou vários neurônios** também conforme a tarefa.

* Em arquiteturas mais avançadas, a rede pode possuir **múltiplas cabeças de saída (*multiple output heads*)**, geralmente compartilhando uma estrutura comum (*backbone*).

NEURÔNIO ARTIFICIAL



Entradas (x) – Conjunto de valores fornecidos ao neurônio, representando os atributos ou características utilizados no processamento.

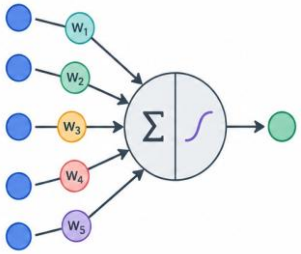
Pesos (w) – Parâmetros aprendidos durante o treinamento que determinam a influência de cada entrada na saída do neurônio.

Bias (b) – Termo adicional aprendido durante o treinamento que ajusta a combinação linear, proporcionando maior flexibilidade ao modelo.

Saída Líquida ou Potencial de Ativação (z) – Resultado da combinação linear entre os valores recebidos da camada anterior, seus respectivos pesos e o *bias*.

$$z = \sum_{i=1}^{n} w_i x_i + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n + b$$

NEURÔNIO ARTIFICIAL



Função de Ativação (f) – Função matemática aplicada à entrada líquida (z) para produzir a saída do neurônio.

- É aplicada sobre a entrada líquida (z).
- Introduz **não linearidade** no modelo, permitindo representar relações complexas entre as variáveis.
- O tipo de função de ativação depende da camada e do problema que se deseja resolver.

Saída (y) – Valor produzido pelo neurônio a partir da entrada líquida (z), após a aplicação da função de ativação, quando utilizada.

- Nas camadas ocultas, constitui uma representação intermediária enviada à próxima camada; **na camada de saída**, corresponde à **resposta produzida pelo modelo**.

Neurônio Artificial aplicado a dados tabulares

1 Dados tabulares

	idade	renda	score	...	alvo (y)
1	25	3000	650	...	0
2	40	8000	700	...	1
3	35	5000	620	...	0
...	⋮	⋮	⋮	⋮	⋮
n	28	4500	680	...	1

Cada linha = uma instância (exemplo)

Cada coluna (exceto alvo) = uma característica

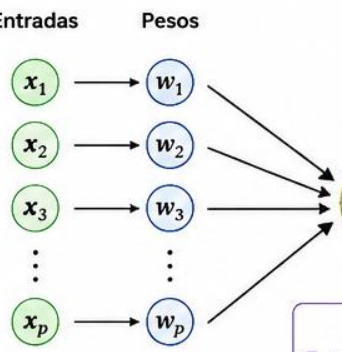
2 Após transformações / padronização

x_1	x_2	x_3	...	x_p	y
-0,45	-1,10	0,20	...	0,35	0
0,85	1,05	1,10	...	-0,15	1
0,30	-0,10	-0,30	...	0,60	0
⋮	⋮	⋮		⋮	⋮
-0,20	-0,60	0,40	...	0,10	1

Cada linha vira um vetor de **entrada**

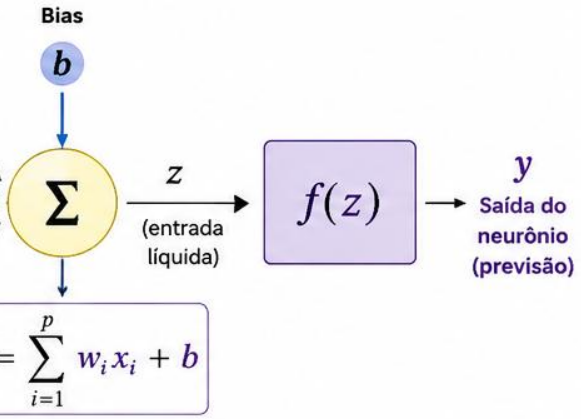
$$x = [x_1, x_2, \ldots, x_p]$$

3 Entradas do neurônio (uma instância)



Cada característica x_i é multiplicada pelo seu peso w_i

4 Soma ponderada + bias



$$z = \sum_{i=1}^{p} w_i x_i + b$$

Soma de todos os produtos mais o *bias* (termo de ajuste)

A função de ativação transforma z na **saída** y

6 Saída é o que queremos prever

y é a previsão gerada pelo neurônio

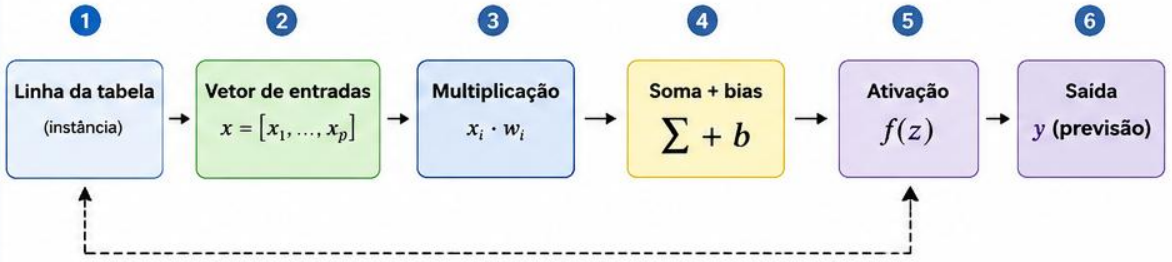
Comparada com o alvo (y real) para calcular o **erro e ajustar** os pesos e o bias.

$\hat{y}$		y (real)
0,12		0
0,87		1
0,23		0
⋮	Comparação (erro/loss)	⋮
0,75		0

Onde cada elemento atua:

- Pesos (w_i): aprendem a importância de cada característica.
- Bias (b): desloca a função, permitindo ajustar o ponto de decisão.
- Função de ativação (f): introduz não linearidade e define o tipo de saída.

Resumo do fluxo de uma instância

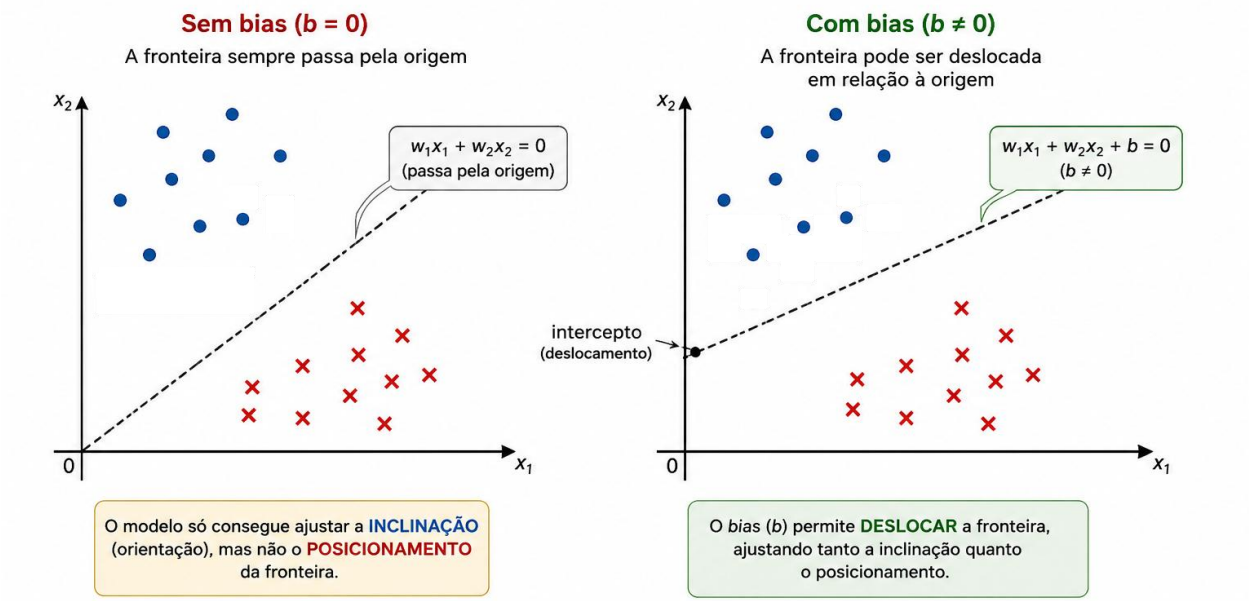


Ajuste dos pesos $(w_1, \ldots, w_p)$ e do *bias* (b) durante o treinamento para reduzir o erro em todas as instâncias

BIAS

O **bias (b)** permite **deslocar a fronteira de decisão** no espaço de entrada em relação à origem.

Sem o bias (**b = 0**), uma fronteira linear definida por $\mathbf{w_1x_1 + w_2x_2 = 0}$ necessariamente passa pela origem. **Com bias** (**b ≠ 0**), a expressão $\mathbf{w_1x_1 + w_2x_2 + b = 0}$ permite deslocar essa fronteira.



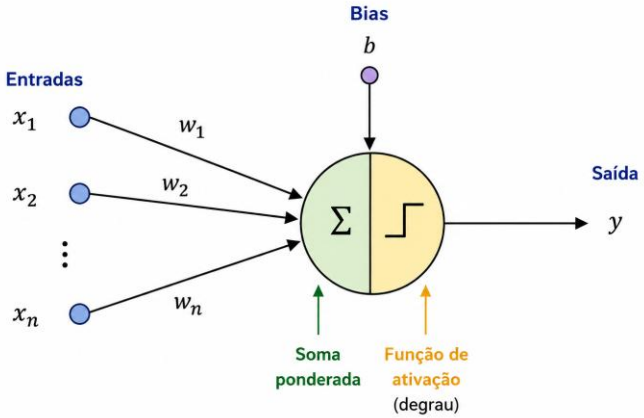


NEURÔNIO ARTIFICIAL
PERCEPTRON CLÁSSICO DE ROSENBLATT

O **Perceptron** foi proposto por **Frank Rosenblatt** em **1958** e é considerado o primeiro modelo de neurônio artificial com **capacidade de aprendizado**. Era uma **rede neural baseada em um único neurônio.**

Embora o **neurônio artificial** seja um conceito mais amplo, o perceptron tornou-se um dos modelos mais importantes na história das redes neurais por introduzir o aprendizado supervisionado por meio do ajuste dos pesos.

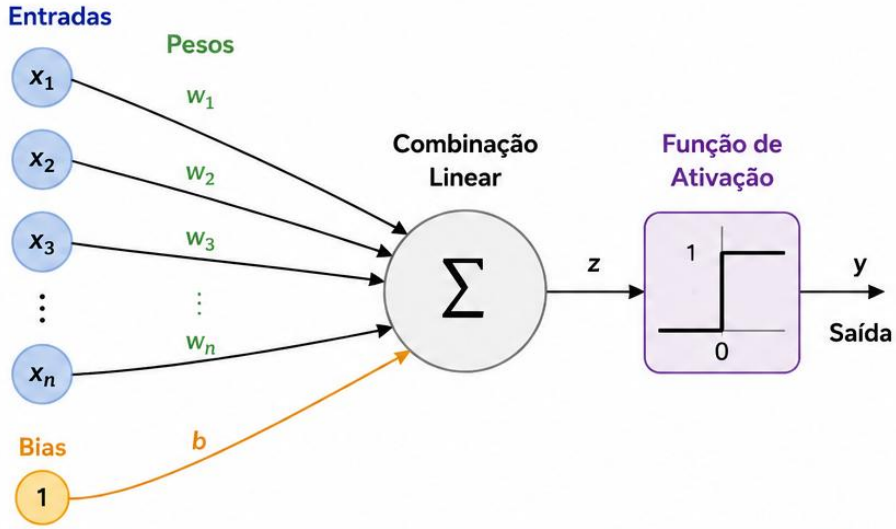
Tradicionalmente, o perceptron é utilizado em problemas de **classificação binária**, nos quais o objetivo é distinguir duas classes, como **0 e 1**, **sim e não** ou **verdadeiro e falso** e usava **função de ativação** degrau (*step* ou *threshold*).





Perceptron Clássico

Modelo mais simples de rede neural artificial para classificação linear binária



Combinação Linear:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

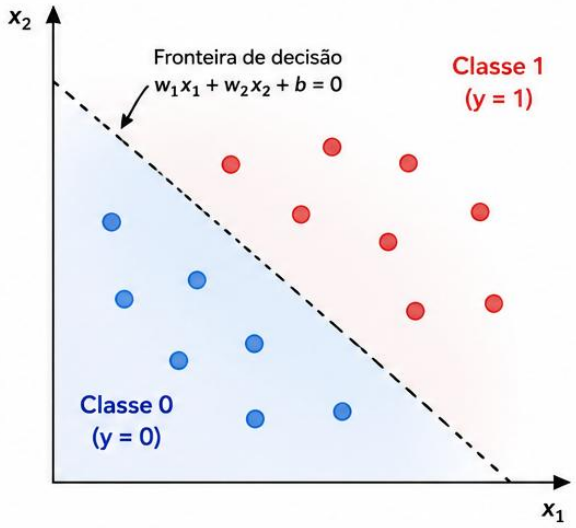
Função de Ativação (degrau):

$$y = f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

Onde:

- x_i = entradas (atributos)
- w_i = pesos
- b = bias (termo de viés)
- f = função de ativação (degrau)
- y = saída (classe prevista)

Separação Linear no Espaço de Entrada (2D)



Interpretação da Saída

- $y = 1$ → instância pertence à Classe 1
- $y = 0$ → instância pertence à Classe 0

Resumo: O perceptron aprende os pesos (w_1, w_2, ..., w_n) e o bias (b) a partir dos dados, de forma que a fronteira de decisão (linear) melhor separe as duas classes.

HIPERPARÂMETROS

Parâmetros

São **aprendidos** pela rede durante o treinamento. Representam **o que** o modelo aprende a partir dos dados.

- Exemplos: **pesos (weights)** e **vieses (biases)**.

Hiperparâmetros

São **definidos pelo cientista de dados**. Determinam **como** e em quais condições o aprendizado ocorrerá.

- Exemplos: número de camadas, número de neurônios por camada, função de ativação, tamanho de lote (*batch size*), número de épocas, função de perda (*loss function*), otimizador e taxa de aprendizado (*learning rate*).

HIPERPARÂMETROS
ARQUITETURA VERSUS TREINAMENTO

Hiperparâmetros de Arquitetura

São paramentos que definem **como a rede neural é estruturada**:

- Número de camadas ocultas;
- Número de neurônios por camada oculta;
- Função de ativação.

Hiperparâmetros de Treinamento

São parâmetros que definem **como o aprendizado acontecerá**:

- Tamanho do lote (*batch size*);
- Número de épocas (*epochs*);
- Função de perda (*loss function*);
- Otimizador;
- Taxa de aprendizado (*learning rate*).

HIPERPARÂMETROS
QUANTIDADE DE CAMADAS OCULTAS

Não existe uma regra que determine o número ideal de camadas ocultas.

A escolha inicial depende principalmente da **complexidade do problema** e deve ser posteriormente **avaliada experimentalmente**.

Em geral, **quanto mais camadas, maior a capacidade da rede de construir representações mais complexas,** mas também **maior o custo computacional e a complexidade do treinamento.**

Na pratica...

- Para problemas tabulares tradicionais, **1 a 3 camadas ocultas** costumam ser um ponto de partida razoável.
- Redes com dezenas ou centenas de camadas estão normalmente associadas a arquiteturas e problemas mais complexos.

HIPERPARÂMETROS
QUANTIDADE DE NEURÔNIOS POR CAMADA

Diferentemente das camadas de entrada e saída, cuja quantidade de neurônios está diretamente relacionada, respectivamente, à dimensionalidade dos dados e ao tipo de problema, **não existe uma regra que determine a quantidade ideal de neurônios nas camadas ocultas**.

A escolha inicial também é uma **hipótese de arquitetura**, posteriormente validada experimentalmente.

Na prática...

- É comum utilizar valores como **32, 64, 128, 256**, por convenção e também por eficiência computacional (legado).
- A arquitetura pode manter, aumentar ou reduzir a quantidade de neurônios entre as camadas. Um **afunilamento progressivo** é uma estratégia bastante utilizada.
- Mais neurônios aumentam a **capacidade de representação** da rede, mas também aumentam o **número de parâmetros e o custo computacional**.

HIPERPARÂMETROS
FUNÇÕES DE ATIVAÇÃO

São aplicadas ao **potencial de ativação (z)** calculado pelo neurônio.

Nas **camadas ocultas***, introduzem **não linearidade**, permitindo que a rede aprenda relações mais complexas (não "linearmente separáveis").

Também é um **hiperparâmetro de arquitetura**, pois diferentes funções de ativação podem afetar significativamente **o processo de aprendizado e o desempenho da rede**.

Entre as principais funções utilizadas estão:

- Sigmoid
- Tanh
- ReLU

* Na **camada de saída**, a função de ativação assume outro papel: sua escolha está relacionada ao **tipo de problema e ao formato esperado para a saída**.

FUNÇÃO SIGMOIDE

$$f(z) = \frac{1}{1 + e^{-z}}$$

A função sigmoide recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **0 e 1**. É uma função **não linear,** contínua e com formato de "S".

É **centrada em 0,5** (para **z = 0,** a saída é **0,5**); para **z ≈ 2,2,** a saída é aproximadamente **0,9**; para **z ≈ -2,2,** aproximadamente **0,1**.

Para valores elevados de |z|, a função entra em uma região de **saturação**, na qual grandes alterações em z produzem pequenas alterações na saída. Essa saturação pode provocar o problema de ***vanishing gradiente*** (gradiente que desaparece), dificultando o treinamento de redes mais profundas*.

* Na retropropagação, para calcular o **gradiente da função de perda em relação aos pesos,** usamos a **regra da cadeia,** que envolve as **derivadas das funções de ativação**. Então a sequência conceitual é: **inclinação da curva → derivada da ativação → cálculo do gradiente → atualização dos pesos.** Por isso a saturação é problemática: uma curva quase horizontal gera derivada ≈ 0 e o gradiente tende a zero, prejudicando a atualização dos pesos (em outras palavras "a rede para de aprender").

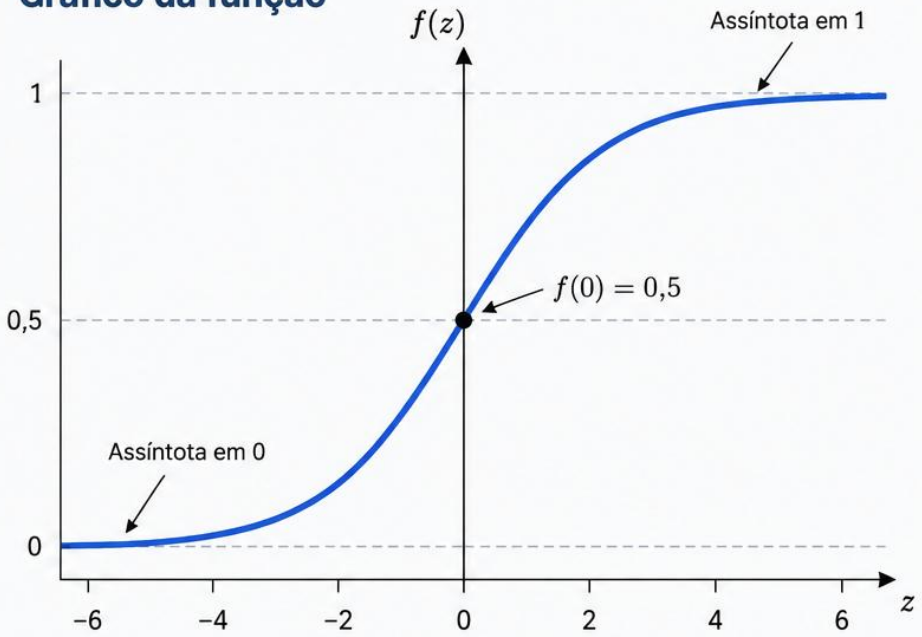
Função Sigmoide

$$f(z) = \frac{1}{1 + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre 0 e 1.
- É uma função não linear, contínua e com formato de "S".
- Para $z = 0$, a saída é 0,5.
- Para valores elevados de $|z|$, a função entra em uma região de saturação.
- A saturação pode provocar o problema de vanishing gradient, dificultando o treinamento de redes mais profundas.
- Atualmente, é pouco utilizada em camadas ocultas, sendo amplamente substituída pela ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0,018	0,119	0,500	0,881	0,982

FUNÇÃO TANGENTE HIPERBÓLICA

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

A função tanh recebe qualquer valor de **z entre -∞ e +∞** e produz uma saída entre **-1 e 1**. Também é **não linear**, contínua e com formato de "S".

É **centrada em 0** (para **z = 0**, a saída é **0**); para **z ≈ 1,47**, a saída é aproximadamente **0,9**; para **z ≈ -1,47**, aproximadamente **-0,9**.

Para valores elevados de |z|, também apresenta **saturação**, aproximando-se de -1 ou 1. Ou seja, também está sujeita ao ***vanishing gradient***.

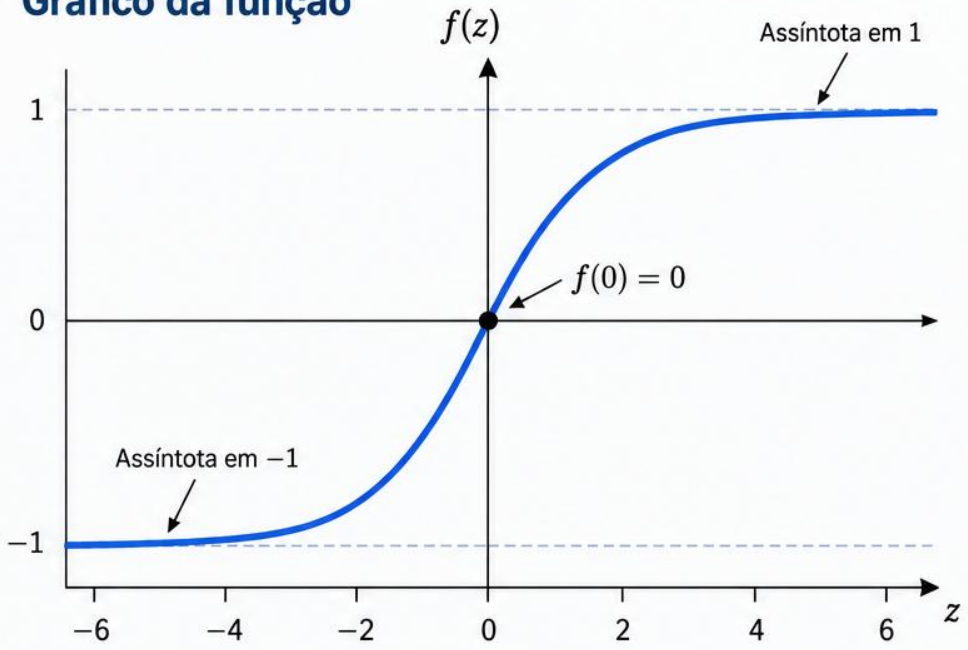
Função Tangente Hiperbólica (*Tanh*)

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Principais características

- Recebe qualquer valor de z (de $-\infty$ a $+\infty$) e produz uma saída entre -1 e 1.
- É uma função não linear, contínua e com formato de "S".
- É simétrica em relação à origem: $tanh(-z) = -tanh(z)$.
- Para $z = 0$, a saída é 0.
- Para valores elevados de $|z|$, a função entra em uma região de saturação (próxima de -1 ou 1).
- Também pode sofrer com o problema de vanishing gradient, embora em menor grau que a função sigmoide.
- Foi amplamente utilizada em redes neurais, sendo hoje menos comum devido ao melhor desempenho da ReLU e suas variantes.

Gráfico da função



Exemplos de valores

z	−4	−2	0	2	4
$tanh(z)$	−0,999	−0,964	0,000	0,964	0,999

HIPERPARÂMETROS
FUNÇÃO RELU

$$f(z) = \max(0, z)$$

ReLU (*Rectified Linear Unit*) é atualmente uma das funções de ativação mais utilizadas em **camadas ocultas**.

Seu comportamento é extremamente simples:

- Para **z ≤ 0**, a saída é **0**;
- Para **z > 0**, a saída é o **próprio z**.

Apesar de sua simplicidade, é **não linear**, pois apresenta uma mudança de comportamento em **z = 0**. Para **z > 0**, sua derivada é **1**, evitando a saturação observada na sigmoide e na tanh nessa região.

Isso contribui para reduzir o problema de ***vanishing gradient*** e facilita o treinamento de redes profundas.

* Como limitação, para **z < 0**, sua derivada é **0**, podendo ocorrer o problema conhecido como ***dying ReLU***. Por isso surgiram variantes como **Leaky ReLU**, que deixam uma pequena inclinação também para valores negativos.

ReLU — Rectified Linear Unit

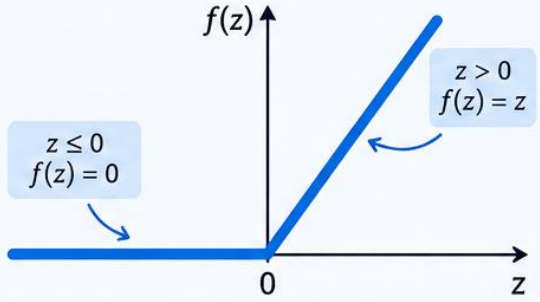
(Unidade Linear Retificada)

- Uma das funções de ativação mais utilizadas em **camadas ocultas.**
- Simples e eficiente.
- É **não linear**, pois muda o comportamento em z = 0.
- Para z > 0, sua derivada é 1, evitando a saturação e ajudando a reduzir o problema de **vanishing gradient.**

Definição

$$f(z) = \max(0, z)$$

Gráfico



Exemplos de valores

z	−4	−2	0	2	4
$f(z)$	0	0	0	2	4
$f'(z)$	0	0	(indef.)	1	1

Obs.: em z = 0, a derivada não é definida. Em implementações práticas, costuma-se usar 0 ou 1.

Vantagens

- Não satura para z > 0 (derivada = 1)
- Ajuda a reduzir o vanishing gradient
- Computacionalmente simples e eficiente
- Funciona bem em redes profundas

Limitações

- Derivada = 0 para z < 0
- Pode causar o problema de "dying ReLU" (neurônios que param de ativar)

Em resumo: a ReLU é simples, eficiente e evita a saturação na região positiva, o que facilita o treinamento de redes profundas. Por isso, é hoje a função de ativação mais usada em camadas ocultas.

HIPERPARÂMETROS
REFLEXÃO

"Em Ciência de Dados, testar é tão importante quanto teorizar.

Muitas descobertas importantes vieram primeiro da observação empírica e posteriormente receberam explicações teóricas."